# Пособие для подготовки к ЕГЭ по информатике 2026

Задания 1-8, 10-11, 14-16 от Алины Лисянской

## Содержание

**Предисловие** **1**

**Задание 1 — Анализ таблиц и схем** **2**

**Задание 2 — Логические выражения. Алгебра логики** **2**

**Задание 3 — Поиск в таблицах и БД** **3**

**Задание 4 — Условие Фано** **4**

**Задание 5 — Алгоритмы** **5**

**Задание 6 — Черепашка** **5**

**Задание 7 — Кодирование изображений и звука** **8**
- Единицы измерения информации . . . 8
- Кодирование изображений . . . 8
- Кодирование звука . . . 9

**Задание 8 — Комбинаторика и системы счисления** **9**
- Системы счисления . . . 9
- Что нужно знать в Python для этого задания . . . 11

**Задание 9 — Работа с таблицами** **12**

**Задание 10 — Поиск слов в тексте** **12**

**Задание 11 — Равномерное кодирование текстовой информации** **12**

**Задание 14 — Кодирование чисел. Системы счисления.** **13**
- Перевод из десятичной системы счисления в произвольную . . . 13
- Перевод из произвольной системы счисления в десятичную . . . 14

**Задание 15 — Алгебра логики** **15**

**Задание 16 — Функции. Рекурсия** **16**
- Модуль sys . . . 16
- Длинная арифметика . . . 16
- Декоратор lru_cache . . . 16

## Предисловие

Это пособие для подготовки к ЕГЭ по информатике. Оно поможет вам систематизировать знания, разобрать задания с неочевидными решениями и освоить приёмы работы в Python, которые реально ускоряют написание кода на экзамене. Материал ориентирован на тех, кто уже знаком с синтаксисом языка и хочет уверенно чувствовать себя при решении задач.

# Задание 1 — Анализ таблиц и схем

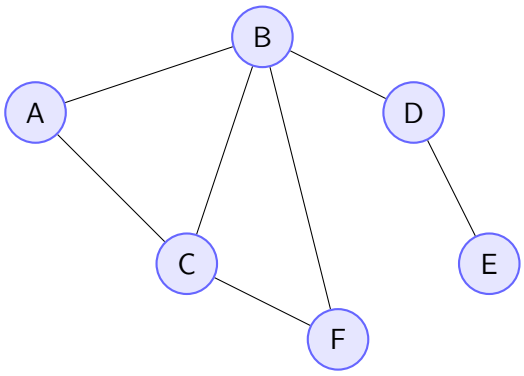
Задание №1 из ЕГЭ по информатике — это задание на графы, где проверяется умение анализировать логические связи.

**Граф** — это структура, моделирующая набор объектов (вершин или узлов) и связи (рёбра) между ними.

Неориентированный граф может быть симметричным и ассиметричным. Асимметричные графы для решения чаще проще, чем симметричные. Для решения симметричных находим разницу между вершинами. Разница в вершинах всегда в степени* и соседями, с которыми эта вершина соединяется.

***Степенью (или порядком) вершины** называется количество рёбер, которые выходят из этой вершины.

Также часто графы в этих заданиях взвешенные. **Взвешенный граф** — это граф, каждому ребру которого поставлено в соответствие некоторое число. В данной задачи это число — длина дороги. Но также в теории это число может называться весом, стоимостью или пропускной способностью ребра (в зависимости от контекста).




# Задание 2 — Логические выражения. Алгебра логики

Задание №2 в ЕГЭ — это задание на алгебру логики. Алгебра логики служит связующим звеном между математикой и языком, поскольку позволяет формально описывать высказывания, их истинность и логические связи между ними с помощью строгих математических правил и символов. В первую очередь для выполнения этого задания нужно ознакомиться с логическими операциями. Во вторую с кодом-шаблоном ниже.

## Таблицы истинности логических операций

### Логическое сложение: «или $A$, или $B$», (дизъюнкция, $A \vee B$), `or` в Python

| $A$ | $B$ | $A \vee B$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

### Логическое умножение: «и $A$, и $B$», (конъюнкция, $A \wedge B$), `and` в Python

| $A$ | $B$ | $A \wedge B$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**Импликация: «Если $A$, то $B$», ($A \to B$), <= в Python**

| $A$ | $B$ | $A \to B$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**Отрицание: «НЕ $A$» ($\neg A$), `not` в Python**

| $A$ | $\neg A$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

**Эквивалентность: «$A$ равнозначно $B$», ($A \equiv B$), == в Python**

| $A$ | $B$ | $A \equiv B$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

### Пример кода на Python

```python
print("x y w z")
for x in range(0, 2):
    for y in range(0, 2):
        for w in range(0, 2):
            for z in range(0, 2):
                if ((not x or y) and (w == z)):
                    print(x, y, w, z)
```

## Задание 3 — Поиск в таблицах и БД

Пожалуй, одна из самых простых задач на ЕГЭ, но всё же коротко обсудим алгоритм решения.

**Пример задачи на БД:** клик на Яндекс.Учебник

При разборе подобных задач необходимо чётко зафиксировать пять ключевых параметров:

1. **Наименование и вид продукта.** В данном случае — зефир всех видов.

2. **Тип операции.** Поступление, продажа или остаток? В данном случае — поступление.

3. **Адрес.**

4. **Временной период.**

5. **Что требуется определить и в какой единице измерения?** Здесь требуется найти массу зефира в килограммах, тогда как в таблице масса зефира указана в граммах!

После того как все параметры определены, настраиваем **автофильтрацию таблицы** по каждому из условий, выбираем соответствующие строки (а точнее убираем ненужные) и подсчитываем необходимое (здесь — суммируем значения в столбце массы).

# Задание 4 — Условие Фано

Задание №4 — это задание на **условие Фано**, которое звучит так: **ни одно кодовое слово не является началом другого**.

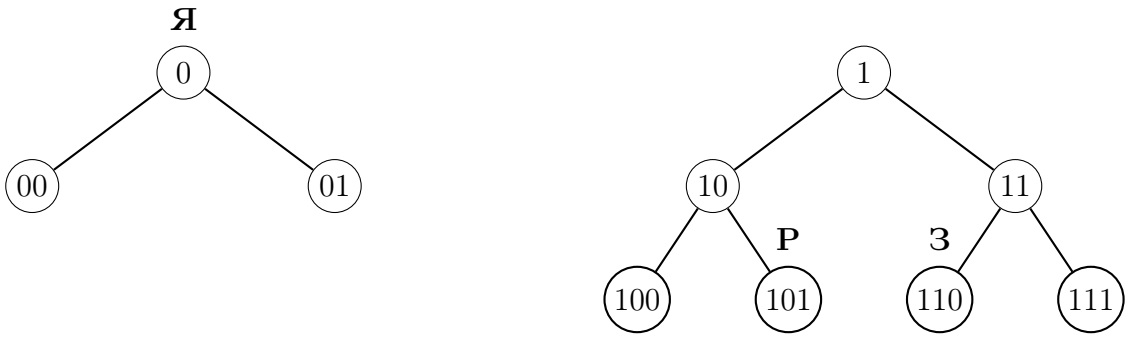
**Пример задачи для ознакомления с заданием:** клик на Яндекс.Учебник

**Алгоритм решения для всех задач №4:**

Строим два двоичных дерева: с корнем 0 и с корнем 1. В каждом дереве левая ветвь добавляет к текущему коду `0`, правая — `1`.

Отмечаем занятые узлы и ищем все свободные узлы, не нарушающие условие Фано:

- узел не должен находиться на пути к уже занятому узлу;
- ни один занятый узел не должен находиться на пути к этому узлу.




Выбираем среди свободных узлов **самый короткий по длине код**. То есть для данной задачи это код `100` или `111`. Но в силу условия указать код с наибольшим числовым значением ответом будет `111`.

**Самые частые ошибки:**

- Не обратить внимание на условие кодирования ВСЕХ букв русского алфавита или наличия «лишних» (тех, что не нужны для составления слова/ ответа). Иначе говоря, не забываем оставить ветвь дерева для кодирования «ненужных» букв.
- При условии составить слово минимальной длины по типу `ВСЕЛЕННАЯ`, не забываем выбрать для самых часто встречаемых букв в слове (тут `Е` и `Н`) самые короткие двоичные коды.

# Задание 5 — Алгоритмы

Самое важное здесь понять алгоритм и его действие. Алгоритмы здесь связаны с системами счисления: с двоичной, с десятичной с операциями умножениями и сложениями цифр в числе и с любой другой (чаще всего троичной). *С теорией по системам счисления можно ознакомиться в задании 8.

## Важные функции на Python для этого задания

`bin(n)[2:]` — перевод числа $n$ в двоичную систему

`oct(n)[2:]` — перевод числа $n$ в восьмеричную систему

`hex(n)[2:]` — перевод числа $n$ в шестнадцатеричную систему

`int(r,b)` — перевод числа $r$ из системы с основанием $b$ в десятичную

## Пример задания на двоичную СС

**Условие задачи:** клик на Яндекс.Учебник

```python
ans = []
for N in range(1, 1000):
    a = bin(N)[2:]
    if N % 2 == 0:
        b = '1' + a + '0'
    else:
        b = '11' + a + '11'
    R = int(b, 2)
    if R > 225:
        ans.append(r)
print(sorted(ans)[0])
```

**Обратите внимание** на обработку результатов и вывод в данной программе. Как видно, алгоритм существенно различается для чётных и нечётных чисел: если для чётных чисел добавляется всего два двоичных разряда, то для нечётных — целых четыре. Из-за этого получаемые значения R не образуют непрерывной последовательности. Следовательно, чтобы найти минимальное подходящее $R$ (или максимальное для другой задачи), необходимо собрать все такие значения в список, отсортировать его и вывевсти минимальное (или максимальное значение) через индекс.

# Задание 6 — Черепашка

Задание №6 в ЕГЭ по информатике — графическое задание, в котором исполнитель **«Черепашка»** перемещается по координатной плоскости, прочерчивая хвостом линии и образуя геометрические фигуры (как правило, прямоугольники или треугольники).

Для успешного решения необходимо владеть библиотекой `turtle` и знать её основные команды. Однако ключевую роль играет **анализ алгоритма**: простой подсчёт точек зачастую неэффективен, и правильный ответ требует выявления закономерностей в траектории движения черепашки.

**Ознакомимся с основными командами Черепашки:**

| Команда | Описание |
|---|---|
| **Экран и отображение** | |
| `screensize(w, h)` | Размер экрана (ширина, высота) |
| `tracer(0)` | Мгновенная прорисовка (без анимации) |
| `update()` | Отобразить всё нарисованное |
| `done()` | Задержка для просмотра результата |
| **Движение Черепашки** | |
| `forward(s)` | Движение вперёд на $s$ шагов |
| `backward(s)` | Движение назад на $s$ шагов |
| **Поворот головы Черепашки** | |
| `right(a)` | Поворот на $a$ градусов вправо |
| `left(a)` | Поворот на $a$ градусов влево |
| **Хвост** | |
| `up()` | Поднять хвост (не рисовать) |
| `down()` | Опустить хвост (рисовать) |
| **Заливка** | |
| `begin_fill()` | Начать заливку контура |
| `end_fill()` | Прекратить заливку контура |
| **Цвет** | |
| `pencolor(color)` | Установка цвета пера (линий) |
| `fillcolor(color)` | Установка цвета кисти (заливки) |
| **Прорисовка плоскости** | |
| `goto(x, y)` | Перемещение в координату $(x, y)$ |
| `dot(d, color)` | Рисует точку диаметром $d$ и цветом `color` |

**Важные моменты:**

- Черепашка по умолчанию смотрит на ось абцисс, то есть вправо (0°). В задачах же голова черепашки сразу направлена по оси ординат, поэтому не забываем повернуть ей голову влево на 90°.
- Чтобы не мучаться из-за мелкого изображения, ставьте масштаб и не забывайте умножать на него движение.
- Обращайте внимание на вопрос в задаче. Что вас просят найти: объединение фигур или их пересечение? И включены ли точки на линиях.

## Пример задания на Черепашку

**Условие задачи:** клик на Яндекс.Учебник

```
from turtle import *
m = 20 #scale
tracer(0)
screensize(3000, 3000)
down()
```

```
left(90)
begin_fill()
fillcolor('red')

for i in range(3):
    forward(7 * m)
    right(90)
    forward(12 * m)
    right(90)
up()
forward(4 * m)
right(90)
forward(6 * m)
left(90)
down()
for i in range(4):
    forward(83 * m)
    right(90)
    forward(77 * m)
    right(90)
up()

for x in range(-100, 100):
    for y in range(-100, 100):
        goto(x * m, y * m)
        dot(3, 'black')
update()
done()
```

В качестве примера намеренно выбрана задача на нахождение объединения двух фигур, притом достаточно протяжённых. Подсчитывать все точки по ширине и высоте каждого прямоугольника вручную крайне трудоёмко (и убийственно для нервной системы), поэтому воспользуемся более элегантным подходом.

**Шаг 1. Связь шагов и точек.** Нам важно осознать, что для единичного отрезка требуется ровно две точки: начальная и конечная. Следовательно, если черепашка прошла $n$ шагов, то на пройденном отрезке располагается $n + 1$ точек. Например, 83 шага — это 84 точки, а 77 шагов — 78 точек.

**Шаг 2. Определение фигур.** Анализируя алгоритм, замечаем, что все повороты при движении Черепашки были на 90°, а значит, черепашка строит прямоугольники. Вычислим количество точек в каждом из них через формулу площади прямоугольника:

$$S_1 = (7 + 1) \times (12 + 1) = 8 \times 13 = 104, \qquad S_2 = (83 + 1) \times (77 + 1) = 84 \times 78 = 6552.$$

**Шаг 3. Область пересечения.** Подсчитаем количество точек в пересечении прямоугольников. Подсчитать это возможно только ориентируясь на полученный рисунок. В данной задаче получим $S_\cap = 7 \times 4 = 28$.

**Шаг 4. Формула включений-исключений.** Точки, лежащие на пересечении двух прямоугольников, при прямом суммировании будут учтены дважды, поэтому при суммировании двух площадей их нужно вычесть (по сути это формула включений-исключений). Количество точек в объединении двух фигур/ итоговый ответ:

$$S_1 \cup S_2 = S_1 + S_2 - S_\cap = 104 + 6552 - 28 = \boxed{6628}.$$

**Лайфхак.** Для задач на треугольник таких элегантных способов нет, но обычно в таком случае точки легко подсчитываются вручную. Лайфхак: сделайте скриншот графика Черепашки, вставьте в Paint и подсчитывайте точки с помощью инструментов для рисования.

# Задание 7 — Кодирование изображений и звука

## Единицы измерения информации

В первую очередь в этом задании стоит ознакомиться с единицами измерения иинформации. Минимальная единица измерения информация — это **1 бит**, который принимает значения либо 0, либо 1. На физическом уровне 1 соответствует высокий уровень напряжения, 0 — низкий.

Не менее важно знание про **байт**. Именно они, а не биты, являются основной единицей хранения и адресации данных.

$$1 \text{ байт} = 8 \text{ бит}$$

Процессор и оперативная память работают не с отдельными битами, а с байтами, где каждый имеет свой адрес. Если бы адресация шла по битам, объём адресов вырос бы в 8 раз, что неэффективно. Далее используются степени двойки.

$$1 \text{ Кбайт} = 1024 \text{ байт}$$
$$1 \text{ Мбайт} = 1024 \text{ Кбайт}$$
$$1 \text{ Гбайт} = 1024 \text{ Мбайт}$$
$$1 \text{ Тбайт} = 1024 \text{ Гбайт}$$

## Кодирование изображений

Как можно понять из названия, в этом задании нужно уметь кодировать изображения и звуки. Важно понимать главное: родной язык компьютера — это биты и байты. Компьютер не понимает, что такое звуки, цвета и картинки. Для него всё — это числа, а точнее — последовательности нулей и единиц.

Любое изображение или звук, прежде чем попасть в память компьютера, должно быть преобразовано в двоичный код. Этот процесс и называется кодированием. Объём изображения в памяти компьютера вычисляется по формуле:

$$V = x \cdot y \cdot b$$

где:

- $x$ — ширина изображения (в пикселях),
- $y$ — высота изображения (в пикселях),
- $b$ — количество бит, отводимых на кодирование одного пикселя.

Количество цветов в изображении определяется формулой: $N = 2^b$

Точно также первоначально объём изображения получается в битах. При необходимости результат переводится в байты, килобайты и т.д.

## Кодирование звука

Объём звукового файла в памяти компьютера вычисляется по формуле:

$$V = f \cdot t \cdot r \cdot c$$

где:

- $f$ — частота дискретизации (Гц/ измерений в секунду),
- $t$ — длительность звучания (в секундах),
- $r$ — разрядность кодирования (в битах),
- $c$ — количество каналов:
  - моно — 1 канал,
  - стерео — 2 канала,
  - квадро — 4 канала.

Точно также как и с изображениями объём звука вычисляется в битах, затем результат при необходимости переводится в байты, Кб и тд.

# Задание 8 — Комбинаторика и системы счисления

## Системы счисления

Системы счисления (СС) — это способ записи чисел с использованием набора цифр и основания.

Пусть основание СС равно $n$. Тогда для записи чисел используются цифры от 0 до $n-1$. Например, в двоичной системе счисления, наиболее важной для компьютерных наук, основание равно 2, а набор цифр состоит из 0 и 1.

Если основание системы счисления превышает 10, то для представления цифр используются латинские буквы:

$$A = 10, \quad B = 11, \quad C = 12, \ \ldots$$

**Перевод в десятичную:** умножаем каждую цифру на основание СС, возведенную в степень разряда (разряды нумеруем справа налево, начиная с нуля), и складываем. Пример:

$$\overset{3\,2\,1\,0}{3645}_8 = 3 \cdot 8^3 + 6 \cdot 8^2 + 4 \cdot 8^1 + 5 \cdot 8^0 = 3 \cdot 512 + 6 \cdot 64 + 4 \cdot 8 + 5 \cdot 1 = 1536 + 384 + 32 + 5 = 1957_{10}$$

**Перевод из десятичной:** последовательно делим число на основание системы, записывая остатки (справа налево).

$$\begin{array}{r|l} 1957 & 8 \\ -1952 & 244 \\ \hline 5 & \end{array} \qquad \begin{array}{r|l} 244 & 8 \\ -240 & 30 \\ \hline 4 & \end{array} \qquad \begin{array}{r|l} 30 & 8 \\ -24 & 3 \\ \hline 6 & \end{array} \qquad \begin{array}{r|l} 3 & 8 \\ -0 & 0 \\ \hline 3 & \end{array}$$

Остатки (снизу вверх): $5, 4, 6, 3$

$$1957_{10} = 3645_8$$

## Комбинаторика

Комбинаторика изучает способы подсчёта количества различных комбинаций, слов и чисел при заданных условиях. В этом задании чаще всего требуется определить количество возможных слов или чисел фиксированной длины.

## Размещения с повторениями

Если используется алфавит из $N$ символов и длина слова равна $k$, причём **повторы разрешены**, то количество различных слов равно: $N^k$

**Пример:** сколько слов длины 4 можно составить из букв А, Б, В?

$$3^4 = 81$$

## Размещения без повторений

Если символы **не повторяются**, то используется формула размещений:

$$A_N^k = \frac{N!}{(N-k)!}$$

**Пример:** сколько слов длины 3 можно составить из букв А, Б, В, Г без повторений?

$$A_4^3 = 4 \cdot 3 \cdot 2 = 24$$

## Перестановки

Перестановки — это размещения всех $N$ элементов без повторений. Количество перестановок вычисляется по формуле:

$$P_N = N!$$

**Пример:** сколькими способами можно переставить 5 различных букв?

$$5! = 120$$

## Метод дополнения

Если требуется найти количество слов, содержащих хотя бы один заданный символ, удобно использовать метод дополнения:

$$\text{Всего вариантов} - \text{варианты без нужного символа}$$

**Пример:** сколько 5-значных чисел в 6-ричной системе счисления содержат хотя бы одну цифру 0?

**Решение**
В 6-ричной системе счисления используются цифры $0, 1, 2, 3, 4, 5$. Так как в условии присутствует формулировка *«хотя бы одна»*, используем метод дополнения.

**Общее количество чисел:**

$$N_{\text{всего}} = 5 \cdot 6^4,$$

так как помним, что число не может начинаться с нуля!

**Количество чисел, не содержащих цифру 0:**

$$N_{\text{без } 0} = 5^5.$$

**Получаем ответ:**

$$\boxed{5 \cdot 6^4 - 5^5}$$

## Что нужно знать в Python для этого задания

```
from itertools import product
```

Подключение заголовка: перебор всех комбинаций с повторениями (слова фиксированной длины).

```
from itertools import permutations
```

Подключения заголовка: перебор размещений без повторений (порядок важен).

```
all(condition for x in iterable)
```

All проверяет, что условие истинно для **всех** элементов последовательности.

```
zip(word, word[1:])
```

Zip объединяет соседние элементы последовательности в пары.

## Примеры решений кодом. Задача на упорядоченный список слов

**Условие задачи:** клик на Яндекс.Учебник

```
from itertools import product

def task8():
    letters = ['Д', 'Е', 'К', 'Н', 'С', 'Я']
    pos = 1

    for word in product(letters, repeat=6):
        if ''.join(word) == 'ЯНДЕКС':
            return pos
        pos += 1
    return -1
print(task8())
```

## Задача на подсчёт комбинаций чисел

**Условие задачи:** клик на Яндекс.Учебник

```
from itertools import permutations

count = 0
for word in permutations([0, 2, 3, 4, 5, 6, 7], r = 5):
```

```
    if word[0] != 0 and all(a % 2 != b % 2 for a, b in zip(word, word[1:])):
        count += 1
print(count)
```

### Важные замечания

- Число **не может начинаться с нуля**.
- В задачах на **перестановки** (а не на комбинаторику) при условии «цифра или буква не используется» соответствующий символ **просто исключается из алфавита**.
- В функции `product` длина слова задаётся параметром `repeat`.
- В функции `permutations` длина слова задаётся параметром `r`.

## Задание 9 — Работа с таблицами

## Задание 10 — Поиск слов в тексте

**В целом, тут всё просто и интуитивно понятно, единственный совет здесь не забывать:**

- Учитывать часть и главу произведения. При необходимости выделяем и копируем нужные главы `(ctrl + C)`, дальше выделяем `(ctrl + A)` весь текст, удаляем его, и возвращаем, то есть вставляем `(ctrl + V)` скопированное.
- Учитывать регистр слова;
- Учитывать поиск слова как отдельного и/или в составе других слов;
- На всякий случай проверить словом с дефисом.

## Задание 11 — Равномерное кодирование текстовой информации

С теорией для решения этой задачи мы уже познакомились в №7. Здесь тоже самое кодирование, но текста. В задании №11 всегда дана мощность алфавита N (или её просят найти).

**Мощность алфавита N** - это общее количество уникальных символов для создания пароля (серийного номера, идентификатора). Мощность алфавита кодируется по правилу: $|N| <= 2^b$. Собственно, также как изображения, звуки и тп.

Если длина сообщения равна $k$, то объём памяти на пароль:

$$V = k \cdot b \text{ бит.}$$

При наличии дополнительных данных:

$$V_{\text{итог}} = V + V_{\text{доп}}$$

Важно помнить, что символы кодируются в битах. Но сами пароли (серийные номера, идентификаторы) **занимают минимально возможное целое число байт**. Также обращаем внимание на формулировки «не более» и «не менее».

# Задание 14 — Кодирование чисел. Системы счисления.

С системами счисления мы уже познакомились в задании №8. В этом задании навык перевода чисел из десятичной системы счисления в произвольную и обратно необходимо перенести на код. Рассмотрим **первый тип задания №14**.

## Перевод из десятичной системы счисления в произвольную

**Условие задачи:** клик на Яндекс.Учебник

Прежде чем перейти к написанию кода, вспомним алгоритм перевода десятичного числа в другую систему счисления на примере $132_{10} \to 7$.

$$\begin{array}{r|l} 132 & 7 \\ -126 & 18 \\ \hline 6 & \end{array} \qquad \begin{array}{r|l} 18 & 7 \\ -14 & 2 \\ \hline 4 & \end{array} \qquad \begin{array}{r|l} 2 & 7 \\ -0 & 0 \\ \hline 2 & \end{array}$$

Остатки (снизу вверх): 6, 4, 2, следовательно:

$$132_{10} = 246_7$$

То есть пока число не равно нулю, фиксируем остаток от деления на основание системы счисления, затем делим нацело. По окончании записываем остатки в обратном порядке. Переведем этот алгоритм в код.

```
num = 132
s = ''
while num != 0:
    s += str(num % 7)
    num //= 7
s = s[::-1]
print(s)
```

Применим разобранный алгоритм к реальному заданию. Принципиальное отличие от учебного примера — масштаб числа: оно намеренно подобрано таким образом, чтобы вычисление вручную было практически невозможным.

```
res = []
for x in range(1, 2030 + 1):
    num = 7**170 + 7 **100 - x
    s = ''
    while num != 0:
        s += str(num % 7)
        num //= 7
    s = s[::-1]
    if s.count('0') == 71:
        res.append(x)
print(max(res))
```

Как и в задании №5, мы не будем выводить все ответы, заполняя терминал, а соберем их в список и выведем с помощью встроенной функции наибольшее число.

**Второй тип задания** — перевод числа из десятичной системы счисления в систему с основанием больше 10. Проблема стандартного алгоритма в том, что остатки, превышающие 9, должны записываться буквами, а не числами — иначе, перевернув строку в конце,

мы получим некорректный результат. Например, остаток 11 в шестнадцатеричной системе — это `b`, а не два отдельных символа `1` и `1`.

**Пример такого задания:** клик на Яндекс.Учебник

Ключевой момент: нас не интересует, какие именно буквы встретятся в записи числа. Важно лишь корректно отличить цифры числа от букв. Поэтому вместо конкретной буквы подставляем любой символ-заменитель, например `*`. Итоговый код получится таким:

```
num = 3*3125**8 + 2*625**7 - 4*625**6 + 3*125**5 - 2*25**4 - 2025
s = ''
while num != 0:
  if num % 25 > 9:
    s += '*'
  else:
    s += str(num % 25)
  num //= 25
s = s[::-1]
print(s.count('0'))
```

## Перевод из произвольной системы счисления в десятичную

Осталось разобрать **третий и заключительный тип** задания №14.
**Пример задания данного типа:** клик на Яндекс.Учебник

В этом задании требуется выполнить обратную операцию — перевести число из произвольной системы счисления в десятичную. В Python для этого существует встроенная функция `int(num, base)`, где `num` — строковое представление числа, а `base` — основание системы счисления.

Особенность задания состоит в том, что одна из цифр числа неизвестна. При большом основании системы счисления цифры обозначаются буквами латинского алфавита, и перечислять их вручную трудоёмко и чревато ошибками. Воспользуемся модулем `string`. На примере задачи нам требуется система счисления с основанием 29 (то есть допустимы 10 цифр и 19 букв). Используем срез строки, чтобы получить ровно нужный набор символов:

```
import string
print(string.ascii_letters[:19])
```

Обсудив все нужные функции и модули, напишем код для задачи:

```
for x in '0123456789abcdefghijklmnopqrs':
  t = int('923' + x + '874', 29) + int('524' + x + '6152', 29)
  if t % 28 == 0:
    print(t // 28)
```

Или проще, объединив эти два кода вместе:

```
import string
digits = string.digits + string.ascii_letters[:19]
for x in digits:
  t = int('923' + x + '874', 29) + int('524' + x + '6152', 29)
  if t % 28 == 0:
    print(t // 28)
```

**Важный момент.** Ознакомьтесь со следующим заданием. клик на РешуЕГЭ

Задача аналогична разобранной выше, однако содержит два важных нюанса:

- Число начинается с цифры $x$, а значит $x \neq 0$ — ведущий ноль в записи числа недопустим.
- Оба числа заданы в разных системах счисления, но неизвестные $x$ и $y$ присутствуют в каждом из них. Следовательно, $x$ и $y$ должны быть допустимыми цифрами в *обеих* системах одновременно — то есть принадлежать наименьшей из них (12-й), чтобы не возникало конфликта интерпретации символов.

Попробуйте решить это задание самостоятельно, а затем сверьтесь с решением:

```
for x in '123456789ab':
    for y in '0123456789ab':
        t = int(x + '231' + y, 12) + int('78' + x + '98' + y, 14)
        if t % 99 == 0:
            print(t // 99)
```

# Задание 15 — Алгебра логики

С теорией в теме алгебры логики мы уже познакомились в задании №2. `Повторить теорию.` К уже полученным знаниям нужно освоить следующий алгоритм решения:

1. «Переворачиваем» результат нашего логического выражения (например, с истины на ложь) и анализируем, при каких значениях переменных такой результат вообще мог получиться.
2. Далее последовательно рассматриваем выражение, двигаясь от самой простой скобки к более сложным.
3. Когда доходим до самой последней скобки (она всегда содержит переменную $A$, значение которой требуется найти), «переворачиваем» её результат и получаем искомый ответ.

Это задание встречается на графики, отрезки, арифметические операции и битовые операции. Для работы с последним типом 15 задания нужно понимать как работает поразрядная конъюнкция. Поразрядное «умножение» работает следующим образом: если оба бита равны 1, которые умножаются друг на другу, результат 1, иначе — 0.

Перед выполнением операции числа переводятся в двоичную систему счисления, после чего операция применяется к каждому биту отдельно.

**Пример:**

$$6_{10} = 110_2, \quad 3_{10} = 011_2$$

$$\begin{array}{r} 110_2 \\ \& \ 011_2 \\ \hline 010_2 \end{array}$$

Затем переводим получившийся результат в десятичную систему счисления:

$$010_2 = 2_{10}$$

# Задание 16 — Функции. Рекурсия

Рекурсия — это поведение функции, при котором она вызывает сама себя. В отличие от цикла, функция не просто повторяется несколько раз — функции работают «внутри» друг друга.

Данное задание мы можем выполнить как вручную, так и с помощью кода, но помимо знания функций в Python, нужно познакомиться с важными встроенными модулями.

## Модуль sys

Чтобы программа не работала бесконечно, в Python существует ограничение на максимальное количество рекурсивных вызовов (примерно 1000). Однако для выполнения задания 16 может потребоваться бóльшая глубина рекурсии. В таком случае это ограничение можно увеличить с помощью встроенного модуля sys, который управляет глубиной рекурсией и завершением программы.

```
import sys
sys.setrecursionlimit(n)
```

## Длинная арифметика

Процессор не может напрямую работать с очень большими (длинными) числами. Если число слишком длинное, то работать с ним нужно как с массивом данных, обрабатывая по частям. В Python существует ограничение на максимальную длину числа при его преобразовании в строку — около 4300 цифр. Если это ограничение превышено, возникает ошибка.

```
ValueError: Exceeds the limit (4300 digits) for integer string
   conversion; use sys.set_int_max_str_digits() to increase the limit
```

Подробно изучать тему длинной арифметики в рамках данного задания не требуется. Однако важно знать о её существовании, чтобы понимать возможные ошибки в программе.

В целом, в заданиях 16 типа ЕГЭ, если алгоритм реализован корректно, проблема длинной арифметики не возникает. Если же вы видите подобную ошибку, это означает, что на выходе получилось слишком большое число. В примерно 95% случаев причина связана с неверно расставленными скобками или ошибкой в логике вычислений.

## Декоратор lru_cache

Модуль `functools`, с которым мы знакомы из задания 8, содержит инструменты для работы с функциями. Один из таких инструментов — декоратор `lru_cache`. Декоратор `@lru_cache` используется для запоминания результатов работы функции, то есть для кэширования промежуточных результатов рекурсии. Благодаря этому при повторном вызове функции с теми же аргументами результат не вычисляется заново, а берётся из памяти, что позволяет ускорять работу программы.

```
from functools import lru_cache
@lru_cache
```

## Пример задачи и кода с данными модулями

**Условие задачи:** клик на Яндекс.Учебник

```
from functools import lru_cache
import sys
sys.setrecursionlimit(10000)
@lru_cache

def f(n):
    if n >= 2025:
        return 1
    else:
        return n - f(n + 2) - f(n + 4)
print(f(20) + f(25))
```

*Попробуйте скомпилировать данный код с данными заголовками и без, и ознакомьтесь с ошибками.